

Sistema de Armazenamento e Recuperacao

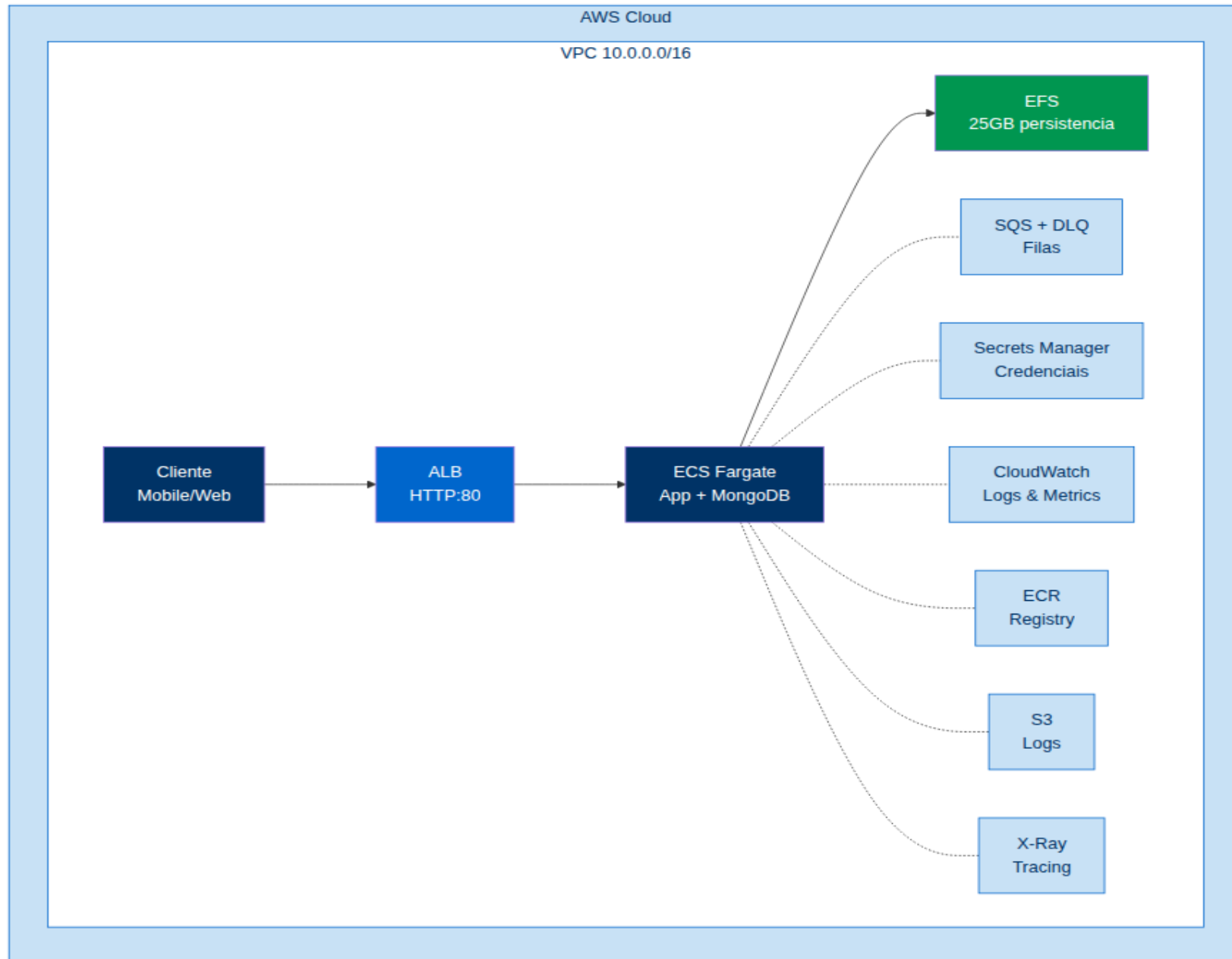
Proposta de Solucao V2 - Disaster Recovery, FinOps e Escalabilidade

Case Tecnico - Engenharia de Software Backend

Itau Unibanco

■ Topicos Abordados na Solucao Proposta (V2)

- 1. Diagnostico e Arquitetura Atual (V1) - Stack e limitacoes
- 2. Diagrama da Arquitetura V1 - Visao geral inicial em ECS Fargate
- 3. Escolha NoSQL vs SQL - Justificativa tecnica para adocao do MongoDB
- 4. MongoDB Atlas (Melhoria #1) - Cluster gerido, multi-AZ e DR
- 5. Arquitetura Hexagonal (Melhoria #2) - Evolucao do DDD e isolamento
- 6. Amazon EKS com Auto-Scaling (Melhoria #3) - Kubernetes, Karpenter e Spot
- 7. SLOs Existentes e Propostas V2 - Analise do README e metas de nivel de servico
- 8. FinOps e Calculo de Custos (Melhoria #4) - Custos V1 vs V2 e otimizacao
- 9. AWS CodePipeline (Melhoria #5) - CI/CD nativo AWS e Blue/Green
- 10. Disaster Recovery (Melhoria #6) - Estrategia cross-region e SLOs
- 11. Diagrama da Arquitetura V2 - Fluxo completo em Amazon EKS
- 12. Resumo e Trade-offs - Conclusao e analise de impacto



Stack Atual

Compute

ECS Fargate (2 tasks App + MongoDB) - 512 CPU / 1024 MB

Arquitetura

Domain Driven Design (DDD) - 4 modulos Maven acoplados

Observability

CloudWatch + X-Ray - Logs e metricas basicas

Database

MongoDB 7.0 auto-gerido em ECS + EFS (Service Discovery)

CI/CD

GitHub Actions - Deploy via script shell manual

DR

EFS como persistencia - Sem estrategia formal (RTO/RPO n/d)

Problemas Identificados (Riscos e Limitacoes)

- MongoDB auto-gerido: sem backup automatico, sem multi-AZ, RPO > 1h
- ECS Fargate: sem instancias Spot, custo fixo elevado sem auto-scaling eficiente
- Arquitetura DDD acoplada: regras de negocio misturadas com infraestrutura
- Pipeline CI/CD (GitHub Actions): sem blue/green nativo e rollback manual
- Ausencia de estrategia formal de Disaster Recovery (RTO/RPO nao definidos)
- FinOps nao implementado: falta de controle, alertas e otimizacao de custos

Por que MongoDB (NoSQL) ao inves de Bancos Relacionais (SQL)?

Flexibilidade de Schema e Documentos JSON

Transacoes financeiras e eventos de contas (AccountCreated, TransactionAuthorized) possuem estruturas dinamicas e aninhadas. O formato de documento do MongoDB acomoda payloads complexos sem necessidade de joins custosos ou migracoes de schema complexas.

Alta Escalabilidade Horizontal (Sharding)

Sistemas bancarios de alta volumetria exigem escalabilidade horizontal nativa. O MongoDB Atlas permite sharding automatico e distribuicao de dados por chaves de particao, superando limites verticais de bancos relacionais tradicionais em instancias RDS.

Performance em Escrita e Alta Disponibilidade

Transacoes por segundo (TPS) elevadas beneficiam-se do motor de armazenamento WiredTiger e de replica sets assincronos/sincronos em 3 AZs. O modelo NoSQL reduz contention de locks comparado a transacoes relacionais pesadas em picos de requisicoes.

Alinhamento com Arquitetura Orientada a Eventos

O consumo assincrono via SQS gera eventos que sao mapeados diretamente para colecoes de documentos. A serializacao nativa JSON/BSON elimina camadas de mapeamento objeto-relacional (ORM) complexas, reduzindo latencia de persistencia.

Por que MongoDB Atlas?

Migracao de MongoDB auto-gerido (ECS + EFS) para MongoDB Atlas M10

O MongoDB Atlas elimina a complexidade operacional de gerir o proprio banco em ECS, oferecendo backup automatico, replicacao multi-AZ, escalabilidade elastica e snapshots para DR.

Caracteristica	MongoDB Auto-gerido (V1)	MongoDB Atlas M10 (V2)
Backup	Manual (script shell)	Automatico (snapshots diarios)
Multi-AZ	Nao	Sim (replicacao em 3 zonas)
RPO	> 1 hora	< 5 minutos
RTO	> 30 minutos	< 5 minutos (failover automatico)
Escalabilidade	Vertical (aumentar task)	Horizontal (sharding nativo)
Manutencao	Operacional (backup, restore)	Zero (gerido pela MongoDB)
Custo	~\$2/mes (EFS + compute)	~\$60/mes (M10)
Seguranca	Basica (senha + SG)	Encryption at rest, LDAP, VPC peering

Estrategia de Disaster Recovery com Atlas

- Snapshots automaticos diarios com retencao de 7 dias (gratuito no M10)
- Restore point-in-time (PIT) com janela de 24h para recuperacao granular
- Replicacao cross-region (sa-east-1 -> us-east-1) para DR ativo-passivo
- Failover automatico em < 2 minutos sem intervencao manual
- Encryption at rest + TLS 1.3 para conformidade LGPD

Evolucao do DDD para Hexagonal Architecture

[X] DDD Atual (V1) - Acoplamento

- transaction-api module depende de infra diretamente
- Controllers chamam repositorios concretos
- Servicos de dominio misturados com infra
- Dificuldade para trocar de banco (MongoDB -> DynamoDB)
- Testes de unidade precisam de mocks pesados

[V] Hexagonal Architecture (V2) - Isolamento

- Core domain isolado (sem dependencias externas)
- Inbound ports: interfaces de entrada (use cases)
- Outbound ports: interfaces de saida (repositorios)
- Adapters concretos na camada de infraestrutura
- Swap de adapters sem impacto no core (ex: MongoDB -> DynamoDB)

Estrutura de Modulos (V2)

transaction-domain (Core)	Entidades, Value Objects, Ports, Domain Events
transaction-application (Inbound)	Use Cases, DTOs, Mappers, Inbound Ports
transaction-infrastructure (Outbound)	Adapters: MongoDB, SQS, S3, CloudWatch, Security
transaction-api (Delivery)	Controllers REST, Exception Handlers, Config Spring Boot

Beneficio: Testabilidade total - core domain testado sem infraestrutura

Evolucao: ECS Fargate -> Amazon EKS com Karpenter + HPA

Caracteristica	ECS Fargate (V1)	Amazon EKS (V2)
Orquestracao	AWS proprietaria	Kubernetes (CNCF)
Auto-scaling	Service Auto Scaling	HPA + Karpenter + CA
Instancias	Sem escolha (Fargate)	Spot, On-Demand, Reserved
Custo compute	~\$5-8/mes (2 tasks)	~\$0-3/mes (Spot, 1 node t3.medium)
Tempo de cold start	~30s (Fargate)	~10s (Karpenter provision)
Portabilidade	Lock-in AWS	Multi-cloud (CNCF)
Ecossistema	Limitado	Helm, Prometheus, Istio, ArgoCD
Plano de controlo	Gratis (Fargate)	\$0.10/hora (~\$73/mes)

Estrategia de Auto-Scaling

HPA

Escala pods por CPU (70%) e memoria (80%). Min 2 pods, max 10.

Karpenter

Escala nos automaticamente. Nos Spot (70% mais baratos) com fallback On-Demand.

Cluster Autoscaler

Aloca pods nao schedulados. Remove nos vazios para otimizar custo.

Pod Disruption Budgets

Garante pelo menos 1 pod disponivel durante atualizacoes.

SLOs Existentes no README.md

SLO	Metrica	Objetivo Atual	Justificativa Operacional
SLO-1: Latencia P95	transaction.authorization.latency.percentile	<= 2s	Transacoes bancarias devem ser rapidas para nao bloquear o fluxo do cliente
SLO-2: Taxa de Falhas	transaction.total.count (status=FAILED)	< 1%	Sistema financeiro 24h precisa de alta confiabilidade nas transacoes
SLO-3: DLQ Messages	ApproximateNumberOfMessagesVisible (DLQ)	0 msg	Mensagens na DLQ indicam falhas assincronas que exigem intervencao
SLO-4: ALB 5xx	HTTPCode_Target_5XX_Count	0 erros	Erros de servidor impactam diretamente a disponibilidade da API REST
SLO-5: SQS Failures	sqs.messages.failed.count	< 0.1%	Falhas no consumo de SQS indicam problemas no processamento de contas

Propostas de Melhoria de SLOs para a Arquitetura V2

SLO-6: Latencia P99 (Proposto V2)

Meta: <= 800ms (reducao de 60% vs V1). Garantido pelo Amazon EKS + Karpenter + MongoDB Atlas M10 com menor latencia de rede e IOPS garantidas.

SLO-7: Disponibilidade (Uptime 99.99%)

Meta: 99.99% de uptime (maximo de 4.38 min de indisponibilidade/mes). Alcançado via multi-AZ no EKS e MongoDB Atlas com failover automatico em < 2 min.

SLO-8: RPO e RTO Automatizados

Meta: RPO < 5 min e RTO < 30 min validados por testes automatizados mensais de restore e failover cross-region (sa-east-1 -> us-east-1).

Monitoramento e Dashboards (CloudWatch)

Dashboard: transaction-api_overview (7 widgets criados via Terraform)

Inclui ECS CPU/Memory, ALB Requests, Business Metrics (transacoes, latencia, saldo), SQS Queue, SLO Compliance, Alarms e Logs.
Link: https://sa-east-1.console.aws.amazon.com/cloudwatch/home?region=sa-east-1#dashboards:name=transaction-api_overview

Comparativo de Custos Mensais Estimados

Componente	V1 (ECS Fargate)	V2 (EKS + Atlas)	Diferenca
Compute (App)	\$5.00 (Fargate)	\$1.50 (Spot t3.medium)	-\$3.50
Compute (MongoDB)	\$3.00 (Fargate)	\$0.00 (Atlas inclui)	-\$3.00
Database	\$2.00 (EFS 25GB)	\$60.00 (Atlas M10)	+\$58.00
ALB	\$0.00 (Free Tier)	\$0.00 (Free Tier)	\$0.00
SQS + DLQ	\$0.00 (1M gratis)	\$0.00 (1M gratis)	\$0.00
CloudWatch	\$0.00 (10GB gratis)	\$0.00 (10GB gratis)	\$0.00
Secrets Manager	\$0.40 (1 secret)	\$0.40 (1 secret)	\$0.00
ECR	\$0.00 (500MB gratis)	\$0.00 (500MB gratis)	\$0.00
EKS Control Plane	\$0.00	\$73.00 (730h)	+\$73.00
X-Ray	\$0.00 (100K traces)	\$0.00 (100K traces)	\$0.00
Pipeline (CI/CD)	\$0.00 (GitHub)	\$1.50 (CodePipeline)	+\$1.50
TOTAL ESTIMADO	~\$10.40/mes	~\$136.40/mes	+\$126.00

Estrategias de Otimizacao FinOps

- Savings Plans: compromisso de 1 ano reduz custo compute em ~30%
- Spot Instances: ate 70% mais barato que On-Demand para workloads stateless
- Auto-scaling baseado em demanda: elimina super-provisionamento
- S3 Lifecycle: STANDARD_IA (30d) -> GLACIER (60d) -> Expire (90d)
- MongoDB Atlas: M10 (2GB) dev, M20 (8GB) staging, M30 (16GB) producao
- CloudWatch: filtros de log para reduzir volume de dados ingeridos

Transicao: GitHub Actions -> AWS CodePipeline



GitHub Actions vs AWS CodePipeline

Aspecto	GitHub Actions	AWS CodePipeline
Integracao AWS	Indireta (OIDC)	Nativa (IAM + ECR + ECS + EKS)
Blue/Green EKS	Script manual	CodeDeploy nativo
Rollback	Custom script	Automatico via CloudWatch Alarms
Custo	Gratis (publico)	~\$1.50/mes
Auditoria	GitHub Audit Log	AWS CloudTrail
Approval Gates	Environments	Manual approval stage nativo
Tempo de deploy	~8 min	~5 min (integracao direta)

Recomendacao: CodePipeline e ideal para times AWS-native

Estrategia de Recuperacao de Desastres

MongoDB Atlas - Replicacao Cross-Region

Replica set em 3 AZs em sa-east-1
Cluster secundario em us-east-1 (DR)
Failover automatico em < 2 min
RPO: < 5 min | RTO: < 30 min

EKS - GitOps com ArgoCD

Cluster EKS multi-AZ (3 subnets)
ArgoCD GitOps: estado no Git
Recuperacao automatica em DR
RTO: < 15 min (aplicacao)

Backup & Restore - Política

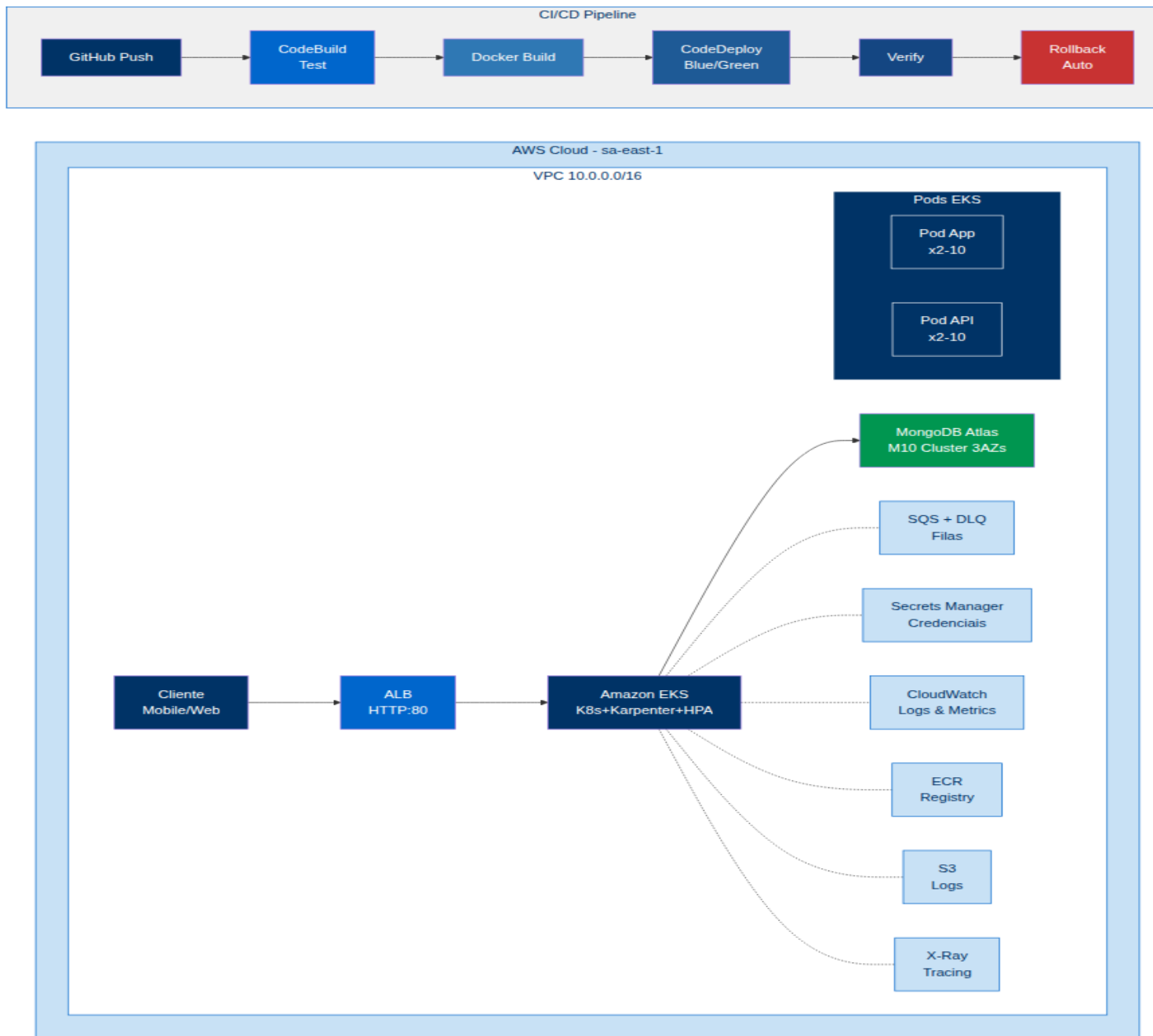
Snapshots Atlas: diarios, 7 dias
PIT Restore: janela de 24h
Backup configs EKS: etcd
Testes de restore mensais

Pipeline de DR - Automacao

CloudWatch -> Lambda -> Restore
Terraform provisionamento DR
Testes de caos (Chaos Monkey)
Runbook documentado no Git

SLOs de Disaster Recovery

SLO	Metrica	Objetivo	Como e Garantido
RPO	Recovery Point Objective	< 5 min	Snapshot automatico Atlas a cada 5 min + WAL archiving
RTO	Recovery Time Objective	< 30 min	Failover automatico Atlas + ArgoCD + Terraform DR
Disponibilidade	Uptime	99.99%	Multi-AZ (3 zonas) + Health checks + Auto-healing
Integridade	Consistencia dados	100%	Transacoes ACID + Validacao pos-restore + Checksums



Resumo das Melhorias Propostas

Componente	V1 (Atual)	V2 (Proposta)	Impacto
Database	MongoDB auto-gerido	MongoDB Atlas M10	[OK] DR, backup, multi-AZ
Arquitetura	DDD	Hexagonal (Ports & Adapters)	[OK] Testabilidade, isolamento
Orquestracao	ECS Fargate	EKS + Karpenter + HPA	[OK] Escalabilidade, Spot
CI/CD	GitHub Actions	AWS CodePipeline	[OK] Blue/Green, rollback
DR	Inexistente	Multi-AZ + Cross-Region	[OK] RPO < 5min, RTO < 30min
FinOps	Nao implementado	Savings Plans + Spot + Lifecycle	[OK] Otimizacao custos

Trade-offs e Riscos

Custo: V2 (~\$136/mes) vs V1 (~\$10/mes)

Aumento justificado por resiliencia, DR, backup nativo e escalabilidade.

Complexidade Operacional: Kubernetes

EKS requer conhecimento K8s. Karpenter, HPA e ArgoCD adicionam complexidade.

Lock-in MongoDB Atlas

Atlas cria dependencia MongoDB. Alternativa futura: DynamoDB via Hexagonal.

Tempo de Migracao

Migracao estimada em 4-6 semanas: Atlas, refactor, EKS, pipeline, testes DR.

Recomendacao Final

A arquitetura V2 e recomendada para ambientes produtivos que exigem alta disponibilidade, recuperacao de desastres e escalabilidade. O custo adicional (~\$126/mes) e compensado pela reducao de risco operacional, conformidade LGPD, e SLOs de RPO < 5 min e RTO < 30 min.